

Manipulations des nombres entiers binaires en C/C++

Eric REMY

eric.remy@univ-amu.fr

Version du 11 octobre 2023

Ce document vise à vous rappeler le fonctionnement des opérateurs logiques « bit à bit » du langage C/C++ et leur utilisation pour consulter ou modifier la valeur de certains bits au sein de variables numériques entières.

Le code source complet de chaque programme d'exemple est disponible grâce au lien Web fourni.

1 Écrire une valeur en octal, hexadécimal ou en binaire

Un nombre écrit sans préfixe particulier en C/C++ est en base 10. Plusieurs préfixes existent pour indiquer une base différente :

- avec le préfixe « 0 », le nombre sera en octal (base 8) ;
- avec le préfixe « 0x », le nombre sera en hexadécimal (base 16) ;
- depuis la norme C++14, avec le préfixe « 0b », le nombre sera en binaire (base 2).

L'exemple ci-dessous montre la mise en œuvre de cette syntaxe pour définir la valeur littérale de quelques variables.

```
1 int var1 = 110; // la variable prend la valeur 11010
2 int var2 = 0110; // la variable prend la valeur 1108 c'est-à-dire 7210 = 1.82 + 1.81 + 0.80
3 int var3 = 0x110; // la variable prend la valeur 11016 c'est-à-dire 27210 = 1.162 + 1.161 + 0.160
4 int var4 = 0b110; // la variable prend la valeur 11016 c'est-à-dire 610 = 1.22 + 1.21 + 0.20
```

Les bits d'un octet sont traditionnellement numérotés du rang 0 (à droite) pour le bit de poids le plus faible vers le rang 7 (à gauche) pour le bit de poids le plus fort.

2 Les opérations booléennes

2.1 Les opérateurs logiques

Vous avez déjà vu en mathématiques et en architecture des ordinateurs les opérations logiques booléennes : NON, ET, OU et OU EXCLUSIF. Les opérateurs logiques équivalents sont en C++ : !, &&, || et ^. Les tables ci-dessous vous rappellent le résultat de chacune de ces opérations.

a	!a	a	b	a && b	a	b	a b	a	b	a ^ b
false	true	false	false	false	false	false	false	false	false	false
false	true	false	true	false	false	true	true	false	true	true
true	false	true	false	false	true	false	true	true	false	true
true	false	true	true	true	true	true	true	true	true	false

2.2 Les opérateurs logiques bit à bit

Vous disposez également d'opérateurs qui appliquent *en colonne* ces opérations logiques sur la représentation binaire de vos variables. On appelle en général ces derniers « opérateurs logiques bit à bit ». Les tables suivantes indiquent le résultat de ces opérations sur les entiers 12 et 3.

a	0b1100=12 ₁₀	a	0b1100=12 ₁₀	a	0b1100=12 ₁₀	a	0b1100=12 ₁₀
!a	0b0011=3 ₁₀	b	0b1010=10 ₁₀	b	0b1010=10 ₁₀	b	0b1010=10 ₁₀
		a&b	0b1000=8 ₁₀	a b	0b1110=14 ₁₀	a^b	0b0110=6 ₁₀

2.3 Décalages

En plus de ces opérations logiques, vous disposez des opérateurs de décalage << et >>. Comme vous le savez, ils servent également, dans un autre contexte, aux opérations de lecture et d'écriture de texte, tout comme l'opérateur * sert à la fois à indiquer la multiplication mais aussi l'opération d'indirection sur un pointeur.

Décaler une variable de n rangs vers la gauche, consiste à décaler les bits de sa représentation binaire de n rangs vers la gauche et à faire rentrer n bits valant zéro à droite. Cela revient à multiplier la valeur de cet entier par 2^n . Sur le même principe, le décalage de n rangs vers la droite, fait rentrer n zéros à gauche (en supposant que c'est un entier

non-signé) et fait disparaître les n bits les plus à droite (les n bits de poids faibles). Un tel décalage est équivalent à la division par 2^n .

Attention, sur des variables dont la taille est limitée, par exemple un `uint8_t` a une taille fixée à 8 bits, les bits qui « sortent » à cause du décalage sont perdus.

Les tables ci-dessous illustrent ces opérations avec un décalage de 3 bits sur des valeurs codées sur 8 bits (`uint8_t`).

a	0b10001100=140 ₁₀	a	0b10001101=141 ₁₀
a<<3	0b01100000=96 ₁₀	a>>3	0b00000011=17 ₁₀

3 Mise en œuvre en C/C++

3.1 Tester un bit

Considérons une variable entière sur 8 bits. Comment tester la valeur d'un bit précis ?

La première étape consiste à utiliser les propriétés de l'opérateur `&` pour faire un *masquage* : faire un ET bit à bit de votre variable `a` avec une valeur où vous ne mettez qu'un seul bit à 1 vous assure de ne conserver inchangé que le bit correspondant dans la variable `a` tout en mettant tous les autres bits à 0. Il ne vous reste ensuite plus qu'à tester avec un `if()` ou un opérateur ternaire si la variable est nulle ou non.

```
1 uint8_t var=...;
2 cout << "Le bit de rang 3 est " << (var & 0b00001000 == 0 ? "nul" : "non-nul") << "." << endl;
```

3.2 Isoler un groupe de bits contigus

Comment isoler les 3 bits de rang 2 à 4 d'une variable de 8 bits pour les considérer comme une valeur entière ?

Il faut masquer uniquement ces bits (les rangs 2, 3 et 4) pour mettre à 0 tous les autres bits en ensuite décaler vers la droite de 2 rangs pour que les bits de rangs 2 à 4 deviennent les bits de rang 0 à 2.

```
1 uint8_t var=...;
2 cout << "Les 3 bits de rangs 2 à 4 font valent " << ((var & 0b00011100) >> 2) << "." << endl;
```

3.3 Regrouper deux valeurs sur 8 bits pour former une valeur sur 16 bits

Soit deux variables entières sur 8 bits `low` et `high`, comment former une valeur entière sur 16 bits ayant les bits de `high` comme poids forts et de `low` comme poids faibles ?

Décalez de 8 rangs vers la gauche la valeur de `high`. Cela fait rentrer 8 zéros à droite et les 8 bits initiaux se retrouvent sur les bits de rangs 8 à 15. Il vous suffit de faire un `|` avec la valeur de `low` pour remplacer les zéros de droite par les bits de `low`.

Évidemment, le principe de ce qui vient d'être montré est généralisable : on peut assembler deux valeurs 16 bits pour former une valeur sur 32 bits, etc.